Day 53 – API Management (APIM) – Part 2

Class date: 10th September 2026 | Module: SAP API Management – Activation & Hands-On Setup

Verified against SAP's own documentation – all accurate. The activation path (Settings → Runtimes → Activate), the four tabs (Test, Configure, Analyze, Engage), and the Inbound/Outbound framing all check out exactly. Organized and lightly expanded below.

1. Recap, and a key clarification: Inbound vs. Outbound (start here first)

Yesterday (Day 52) was a **theoretical session** on API Management: what it is, and where exactly it's used.

Important clarification: API Management is only used for **Inbound** scenarios – **Outbound is "No."** This only makes sense once you fix **whose perspective** you're using, because "inbound" and "outbound" are relative terms.

SAP's perspective (the convention used throughout this course):

- **SAP** → **3rd Party** = **Outbound** (SAP is the one initiating the call out).
- **3rd Party** → **SAP** = **Inbound** (an external party is calling into SAP).

Why this matters: whenever someone talks about "inbound" or "outbound," you need to ask **which perspective they're speaking from** – e.g. SAP's perspective, as used here.

Putting it together: API Management is used when a **3rd party needs to call into SAP** (Inbound, from SAP's perspective) – because that's exactly the scenario where you want to protect your own backend by exposing a secured Proxy URL instead of the real one (as covered on Day 52). A pure Outbound call (SAP calling out to someone else) doesn't need this same protection layer, since CPI itself handles that call directly.

2. How to start with API Management – Activation

Practical, Step 0: first check whether the API Management capability is already enabled on your tenant or not.

Steps to activate API Management:

1. **Step 1:** Add the proper **Roles** for your user (so you have the right permissions to work with the capability).
2. **Step 2:** Check capabilities under the **Monitor** tab.
3. **Step 3:** Go to **Settings** → **Runtimes.**
4. **Step 4:** Find **Classic API Management** and choose **Activate** (the actual button is labelled Activate, functioning as Enable).

Extra tabs that appear after activating: Test, Configure, Analyze, Engage. These sit alongside Settings as the main navigation for the API Management capability.

Where to start building: Configure tab → Configure → APIs.

3. Let's start API Management – the Configure tab

Under **Configure**, the key artifact types are: **API Proxies, API Providers, Certificates, Key Value Maps,** and **Policy Templates.**

API Proxies – what you actually **build** and then let consumers **subscribe to**, e.g. a proxy that connects through to **S/4HANA.**

API Providers – used to **connect to any backend system** (e.g. S/4HANA). This is the piece that tells API Management *where* the real backend actually is.

Certificates – for creating or updating certificates. Worth remembering: **API Management has its own separate Certificates store** (for importing/sharing certs used by API Management itself), which is **different from CPI's own Keystores** (covered back in the security/OAuth days). The two certificate stores are not the same thing, even though both belong to the same Integration Suite tenant.

Key Value Maps – for storing simple key-value configuration data used by policies.

Policy Templates – reusable, pre-built policy configurations (e.g. security, traffic management) that can be applied to an API Proxy.

4. Steps to create an API in API Management

1. **Step 1:** Create an **API Provider.**
2. **Step 2: API Proxy creation** (provide/secure the API created via the Provider).
3. **Step 3:** Creating a **Product.**
4. **Step 4:** Creating an **Application.**

API Provider → **connects to the Backend System** (e.g. S/4HANA).

5. Demo scenario – building towards a Pass-Through flow

API to create: **API_Northwind** (SAP's own sample SFTP/API proxy used for practice is **not for internet** access – it runs **on-premise**, requiring the appropriate **Location ID**, same concept as the Cloud Connector days).

```
Overall scenario: 3rd Party -> API Management -> CPI -> Target
```

Demo, Step 1 – build the CPI side first

- **Design** a new iFlow: **APIDemoFlow.**
- Structure: **Sender (HTTPS, calling /APIDemoFlow)** → **Content Modifier** → **Receiver.** The Content Modifier simply sets a demo message body: *"Welcome to CPI class."*
- **Save and Deploy** the iFlow.

Demo, Step 2 – create the Integration Flow's Service Key

Next, create the integration flow **Plan** and its **Service Keys** – specifically, create an **API Service Key** for it. (This is the same Service Key concept covered back on the OAuth days: Client ID, Client Secret, Token URL – now being generated specifically so API Management can call this particular CPI iFlow.)

Demo, Step 3 – how API Management will call the CPI iFlow

APIM → **CPI iFlow**: this is exactly how API Management will call the Target. The connection path is: **API Provider** → **CPI Connection.**

6. Creating the API Provider – field by field

An API Provider has 4 sections: **Overview, Connection, Catalog Service, Settings.**

Overview tab:

- **Name** – e.g. *CPI Connection*.
- **Description** – free text.

Connection tab – Connection Details:

- **Type:** Internet, On-Premise, Open Connector, or **Cloud Integration** (the type used for connecting directly to a CPI iFlow, which is the one used here).

For Type = Cloud Integration, the fields map directly to values from the **Service Key** you created on BTP Cockpit:

Field	Where it comes from
Host (Tenant URL)	From the Service Key's URL/host value in BTP Cockpit.
Port	443 (default)
Authentication	OAuth2 (API)
Client ID	From the Service Key details.
Client Secret	From the Service Key details.

Finally: **Save**, then run **Test Connection** to confirm API Management can successfully reach the CPI tenant using these Service Key credentials.

7. Now, API Proxy creation

You can create an API Proxy by using an **existing API Provider** – via a dropdown option called **Discover**, which checks for available API-based flows. This confirms: CPI exposes its credentials, API Management connects to our system, and the whole link is an **API-based connection.**

Also checked: an **Internet-type connection** was demonstrated as well, using the public **Northwind** API as the example (host, port **443**), to show that API Proxies aren't limited to CPI-based providers – any reachable API can sit behind a Proxy.

Steps:

1. **Create API Proxy** (based on the chosen Provider).
2. The **API Management Proxy endpoint** gets created.
3. **Test** it – either via **Postman**, or directly using the built-in **Test tab** inside API Management.

8. Gateway scenario / Pass-Through scenario – recap

Tying back to Day 52: in this whole setup, the two key artifacts pass along different credentials:

Artifact	What it provides
API Provider	Supplies the **API/Service Key details** (how API Management authenticates to the real backend, e.g. CPI).

API Proxy	Represents the actual **Integration Flow** being exposed, carrying its own Service Key details for the consumer side.

This is exactly the **Pass-Through scenario** from Day 52: API Management sits in front, handling security/routing, while the underlying CPI iFlow (or backend system) does the actual work – with no transformation happening inside API Management itself.

Quick-recall summary

- **Inbound/Outbound are relative** – from SAP's perspective: 3rd Party → SAP = Inbound; SAP → 3rd Party = Outbound. **API Management is only for Inbound** scenarios.
- Activation: **Roles** → **check Monitor tab** → **Settings** → **Runtimes** → **Classic API Management** → **Activate**, which unlocks the Test, Configure, Analyze, Engage tabs.
- Configure tab artifacts: **API Proxies, API Providers, Certificates (separate from CPI's own Keystores), Key Value Maps, Policy Templates.**
- 4 steps to create an API: **Create API Provider** → **API Proxy creation** → **Create a Product** → **Create an Application.**
- **API Provider** (Type = Cloud Integration) connects using the CPI tenant's own **Service Key** (Host, Port 443, OAuth2 Client ID/Secret).
- **API Proxy** can be built from an existing Provider via **Discover**, then tested via Postman or the built-in Test tab.